ELI MKXI — Full Project Breakdown and Assessment

Contents

1. What ELI is 1
2. Scale & shape 1
3. Architecture, layer by layer 3
4. What's genuinely strong 3
5. Where it's weak (grounded in the sweep) 3
6. Honest verdict on "frontier, ground-breaking" 4
7. Highest-leverage work (the next month) 4

Update Advisory — 2026-06-01 4
Update Advisory — 2026-06-07 5
Update Advisory — 2026-06-07 5
Update Advisory — 2026-06-08 5
Update Advisory — 2026-06-08 (reliability + voice/wake/tone + cognition correctness) 5

A grounded, total-project read: what ELI is, its scale and shape, the architecture layer by layer, what's strong, where it's weak (with numbers), an honest verdict, and the highest-leverage work. Companion to `orchestration_and_agents.md` (agent/bus detail) and `agent_bus.md`.

> Method note: this is built from deep reads of the core modules (engine, executor, router, agent_bus, orchestrator, planners, memory, grounding gate, vision, identity, security, reasoning modes) plus a complete structural and code-health sweep (LOC distribution, debt markers, duplication, repo hygiene). It is grounded in a deep read of every package this session — claims are grounded in what was actually inspected.

---

1. What ELI is

A **100% local, model-agnostic personal AI** — no cloud, no telemetry, no API calls at runtime (one-time opt-in HuggingFace model downloads are the only network event). It bundles: GGUF inference (llama-cpp), a PySide6 desktop GUI, persistent SQLite+FTS5+FAISS memory, a knowledge graph, a 12-stage cognitive pipeline with a parallel multi-agent bus, a deterministic grounding/evidence layer to fight confabulation, local vision (Qwen2.5-VL hot-swap + Moondream co-resident), TTS/STT, OS control, a plugin system, a proactive daemon, and a LoRA self-training loop. It is a cognitive operating system for one machine, not a chat wrapper.

2. Scale & shape

~134,000 LOC across 353 Python files (`eli/`), plus 152 test files. *(2026-06-09.)*

Subsystem	LOC	Files	Role
`execution/`	19.7k	14	router + executor (action dispatch)

Subsystem	LOC	Files	Role
`gui/`	18.3k	18	PySide6 desktop app
`runtime/`	17.3k	66	grounding, evidence, introspection, surfaces
`kernel/`	12.9k	8	the engine (pipeline driver)
`cognition/`	12.2k	26	agent bus, orchestrator, inference, persona, modes
`tools/`	7.9k	27	image engine, news, etc.
`memory/`	6.6k	13	SQLite/FTS5 + FAISS + KG
`perception/`	5.1k	18	vision, STT, TTS, OS control
`core/`	4.2k	15	paths, settings, hardware profile
`learning/`	3.1k	11	LoRA self-training (Phi-3 base)
`planning/`	3.0k	21	proactive daemon, task planning
`plugins/`	1.9k	28	runtime plugin manager
`world/`	1.5k	26	world event bus, local world bridge
`integrations/`,`utils/`,`contracts/`,`system/`,`cli/`	~3k	—	misc

Four files carry ~⅓ of the codebase: executor_enhanced.py (12.1k LOC), engine.py (11.8k), gui/eli_pro_audio_gui_MKI.py (10.3k), router_enhanced.py (6.0k). Next tier: labs_tab.py (5.1k), deterministic_grounding_gate.py (4.3k), memory.py (4.1k).

3. Architecture, layer by layer

- **Boot / hardware** — core/hardware_profile.py + core/startup_hardware_optimizer.py adapt n_ctx / gpu_layers / batch to whatever model + GPU are present (filename→ctx table; VRAM compute-buffer reservation). Model-agnostic.
- **Routing** — execution/router_enhanced.py: regex-first with LLM-intent fallback + an explicit priority pipeline → one of **206 manifest capabilities** (~163 executor SUPPORTED_ACTIONS). Full reference with activation phrases: capabilities_and_actions.md.
- **Orchestration** — kernel/engine.py gates between the 12-stage cognition/orchestrator.py AgentOrchestrator (non-quick modes) and the parallel 14-agent cognition/agent_bus.py AgentBus (quick / fallback). ReAct tool-chaining for non-CHAT actions. Selection now flows through one typed ExecutionPlan (execution/execution_planner.py). See orchestration_and_agents.md for full detail.
- **Inference** — cognition/gguf_inference.py: model-path resolved from env / settings (no baked model); RLock-serialized; live runtime override; hot-swap with vision models.
- **Memory** — memory/memory.py is the shared foundation (FTS5 + FAISS + KG via recall_memory); two deliberate retrieval strategies sit on top (lightweight bus BusMemoryAgent vs heavyweight orchestrator OrchestratorMemoryAgent with HyDE + RAG + rerank). memory/knowledge_graph.py, memory/vector_store.py (FAISS, JSON meta).
- **Grounding / evidence (the crown jewel)** — runtime/deterministic_grounding_gate.py (4.3k), runtime/evidence_ledger.py, runtime/evidence_arbitration.py, runtime/control_contracts runtime/grounded_remediation.py (1.3k), cognition/output_governor.py. A layered, deterministic anti-confabulation system wrapped around the probabilistic model. Most local-LLM projects have nothing comparable.
- **Security** — runtime/security.py SecurityManager: **fail-closed shell gate** (ELI_ALLOWED_CMDS / ELI_FULL_CONTROL; unset ⇒ commands blocked), path allow-roots (ELI_ALLOW_ROOTS, defaults to project + home), app allowlist with a default-safe set, **SHA-256 custom-agent trust registry** (config/trusted_agents.json), prompt-injection guard, SQL identifier validation.
- **Self-model** — cognition/reasoning_modes.py: quick/fast/balanced all fastpath to quick, plus four private modes (chain_of_thought, self_consistency, tree_of_thoughts, constitutional_ai). Persona overlay, runtime/self_improvement.py, LoRA self-training (learning/). Emergent internal-state surfacing (autonomy pressure, "anomaly room") is intentional — see memory eli-emergent-voice.

4. What's genuinely strong

1. **The grounding / evidence layer.** Unusually rigorous; deterministic evidence gating around a probabilistic model is the right idea and well-developed. This is the part closest to genuinely frontier.
2. **Truly local + truly model-agnostic.** No call-home, hardware-adaptive, swappable models (verified — inference path carries no baked model identity).
3. **Real, fail-closed security.** Shell blocked by default, hash-gated custom agents, path/app allowlists. Most "AI agent" projects ship wide open.
4. **Breadth, integrated.** Vision, voice, OS control, memory, KG, plugins, self-training — all local, all wired into one pipeline.

5. Where it's weak (grounded in the sweep)

1. **2,565 `except Exception:` blocks (+7 bare `except:`).** The dominant structural problem. Errors are swallowed into "skipped"/fallback/empty everywhere — which is precisely why bugs surface only via runtime logs. Failures are invisible by design. Frontier software makes failures **loud in dev, quiet in prod**; this swallows them uniformly. Also the main reason the system is hard to reason about. (Only 12 TODO/FIXME markers exist — not because the code is clean, but because failures are absorbed rather than flagged.)

2. **God-files.** Two ~12k-line modules (`executor_enhanced.py`, `engine.py`) plus a 10k GUI. The executor is a giant if/elif action ladder. High regression surface, hard to hold in the head, painful to unit-test.
3. **Duplication & overlap.** Two separate image-engine implementations (`eli/tools/image_engine.py` 1.75k LOC *and* the `eli/tools/image_engine/image_engine/` package). `runtime/` (66 files) has many near-duplicate `personal_memory_*` / `*_surface` / `*_response` modules doing overlapping grounding work. Several plan representations coexisted (partly consolidated). Fingerprint of fast solo iteration — new code added beside the old, not folded in.
4. **Repo hygiene.** Committed junk at root: empty `...` and `[package-index-options]` files, one-off `patch_gpu_dynamic.py` / `patch_s11_bugs.py`, three `verify_eli_claims*.sh` versions, diag outputs, `.coverage`, and `experimental/*.zip` binaries. Makes the repo look less serious than the code is.
5. **Tests are GREEN (measured 2026-06-08).** 151 test files; `pytest tests/` = **6,586 passed / 42 skipped / 2 xfailed / 0 failed** (6,630 collected, ~6m47s on the `.venv`/GPU). (Earlier baselines: 2,356 then 6,371 passing — the suite has grown ~3× with the `tests/claims/` contract layer and is a real safety net.)
6. **13 monkeypatch/`globals()` hacks** — mostly load-bearing (e.g. the CPU-clip vision fix), but they're fragile seams worth tracking.

6. Honest verdict on "frontier, ground-breaking"

In ambition and in specific subsystems — yes. The grounding layer, the fully local model-agnostic design, and the integrated multi-modal local agent are genuinely ahead of most open local-assistant projects. The ideas are frontier-grade.

In engineering discipline — not yet. The 2,565 swallowed exceptions, the god-files, the duplication, and the clutter separate "an extraordinarily ambitious solo project" from "software others can build on." None of that is a vision problem — it is consolidation and observability. The gap to "ground-breaking" is **subtraction and discipline, not more features.**

7. Highest-leverage work (the next month)

Ranked by effect-per-effort:

1. **Tame error-swallowing.** Replace blanket `except Exception: pass`/fallback with scoped exception types + a single structured error log (a ring buffer / table) you can actually watch. Keep the graceful degradation, but record every swallow. This alone makes the whole system debuggable and is the precondition for trusting any other change.
2. **Split the two god-files** along their natural seams (executor: action groups into per-domain modules behind the dispatch table; engine: pipeline stages into stage modules). Reduces regression surface and makes the pipeline readable.
3. **Delete duplication + clutter; get tests green.** Collapse the two image engines, remove root junk/one-off scripts, fix or quarantine the ~24 failing tests. Cheap, high signal-to-noise improvement.
4. **Consolidate the `runtime/` surfaces.** The many `personal_memory_*` / `*_surface` / `*_response` modules want to be a handful of well-named ones.

Do 1–3 and the engineering would match the ideas — which is the only thing standing between ELI and the label "frontier, ground-breaking software people can rely on."

Update Advisory — 2026-06-01

- New since the LOC table was captured: `eli/coding/` (coding agent), `eli/core/dag.py` (DAG engine), `eli/runtime/background_tasks.py`, `eli/gui/coding_tab.py`. Re-run the LOC/file sweep to refresh §2.

- Highest-leverage item #1 (tame the ~2,322 `except Exception`) is STILL OPEN and remains the top priority — the new subsystems add surface that also deserves a structured error log. Items: god-file split + duplication cleanup unchanged.

Update Advisory — 2026-06-07

- **§2 numbers refreshed:** 126,619 LOC / 336 files; biggest files now `executor_enhanced.py` (13.3k), `engine.py` (12.5k), GUI (10.7k), `router_enhanced.py` (6.5k), `labs_tab.py` (5.1k), `deterministic_grounding_gate.py` (4.29k after dead-fragment removal), `memory.py` (4.3k).
- **§5 weaknesses — progress:** 'tests not green' is RESOLVED (eval green + now run under pytest). Duplication is reduced: the 3 governance/normalizer modules are consolidated to `output_governor` (+shims); the dual-DB failure split is unified to `agent.sqlite3`. Still open: the `except Exception` swallowing (now ~2,565) and the god-files.
- **§3 capability surface:** 193 capabilities in the manifest (`capability_sync` keeps it measured, not asserted); ~157 executor actions.

Update Advisory — 2026-06-07

- Counts refreshed: **~130k LOC / 346 files**; capability surface = **200 manifest** (~163 `SUPPORTED_ACTIONS`, 13 plugin-backed); **14 main GUI tabs**.
- **Tests green** (~6,464 passed) on the real `.venv` — the "tests not green" finding is long resolved; the suite now examines the project vs its claims at scale.
- New this session (see per-area advisories): the **DAG execution orchestrator** (`eli/core/dag.py`) wired through agents + evidence gather (`dag_orchestrator.md`); evidence-planner + multi-stage `report_pipeline` for grounded generation; **Test & Review** (full suite → LLM summary → backups/errors → option chat, delegating analysis to the code examiner via `run_graph`) + **Orchestration** — both now **Labs sub-tabs** (developer/diagnostic tools, demoted from top-level 2026-06-18); LoRA pipeline wired + model-agnostic; cross-platform one-click installers + CUDA toolkit option; test monkeypatches removed (clean in-project config); GPU corrected (CUDA active in the venv). Authoritative current state: `state_snapshot.md`.
- Full per-action reference with activation phrases: `capabilities_and_actions.md` (auto-generated by `eli/tools/registry/capabilities_doc.py`, in sync with the manifest).

Update Advisory — 2026-06-08

- Counts: **~131k LOC / 349 files**; **201 manifest capabilities**; **~6,508 tests**.
- **Routing/scheduling**: "do at " → background workers (generalised `schedule_prepass`, runs before app-routing, clean re-run); compound "schedule a morning report for 7:15 tomorrow" now schedules (was run-now); recurring on "every/daily". **Multi-command chaining** (`MULTI_COMMAND`): one utterance chaining imperative commands runs each in order (router+executor level → voice + typed). See `runtime_planning_world.md`, `dag_orchestrator.md`, `state_snapshot.md`.

Update Advisory — 2026-06-08 (reliability + voice/wake/tone + cognition correctness)

- **Scale now:** 133,430 LOC / **352** files / **205** capabilities (166 SUPPORTED_ACTIONS) / 151 test files. New actions: `WAKE_TRAIN` / `WAKE_ENROLL` / `WAKE_SET` / `TRAIN_VOICE`. New subsystems: `perception/wakeword.py`, `perception/voice_profile.py`; rewritten `cognition/llm_intent.py`.
- **Reliability fixes (all tested):** MULTI_COMMAND `UnboundLocalError` (a shadowed `execute` local); FIX_FILE backup crash (shadowed `datetime`); LoRA `build_job` now **builds** the dataset (615 rows, was the recurring ×30 "does not exist") and treats "no curated data yet" as benign; the **vision VRAM cliff** (reload restores the last known-good full-GPU config instead of collapsing to `gpu_layers=16`).

- **Intelligence over hardcodes:** the model-grounded intent resolver now actually runs on unmatched phrasings (the engine gate was dead), pulling factual near-misses into grounded actions — date→DATE, “set the volume to 40%”→VOLUME — with no phrasing regexes.
- **Correctness:** mode-gated determinism fixed (command actions verbatim in quick, synthesised in non-quick — was corrupting “Wrote”→“Bought”); the “LLM bypass” claim corrected to “partial + mode-gated” (see `grounding_and_evidence.md`).
- **Voice/wake/tone:** a **self-trained, music-robust wake word** (Piper-synth + augmentation + openWakeWord features + a custom head), **user-settable phrase**, and a **voice-profile/tone** subsystem (pitch/energy/rate, question-vs-statement, labelled emotion) **wired into cognition** (the engine adapts its delivery to the user’s vocal tone). Duration-adaptive STT pause (0.5s / 2s-after-12s). See `perception.md`.
- **Standing debt (unchanged):** god-files; ~798 silent `except: pass`; routing/verbatim logic duplicated across router/engine/GUI (the source of by-path inconsistencies); the model ceiling. See `decomposition_plan.md`.